

RoadSense AI — PRD Review & Action Plan

Purpose: A living record of our section-by-section walkthrough of the PRD. For each PRD section we capture: (1) the cleaned-up / agreed version, (2) where we actually stand today, and (3) the exact tests or actions needed to close the gap. When the walkthrough is finished, this document is the to-do list to act on.

Companion to `RoadSense_AI_PRD.md` . Status tracker lives in `../CLAUDE.md` .

Started: 2026-07-18 · Sections covered so far: **Goals**

1. Goals

1a. Agreed rewrite (measurable + honest)

Primary goals — what we control and will be graded on:

#	Goal	Target (the number)
1	Keep the data trustworthy — few false alarms	False-positive rate < 5% on real footage
2	Detect & geo-tag road damage automatically	Start with potholes + cracks , expand toward 7 types
3	Merge duplicate sightings into one verified issue	> 80% duplicate reduction
4	Get reports to the cloud fast	Upload in < 5 seconds

Impact goals — why we're doing this (not graded at ship; depend on others acting):

- Help authorities repair roads faster
- Reduce vehicle damage
- Improve public safety
- Predict road degradation over time

Why this differs from the original PRD:

- Every primary goal now has a number → provable.
- Split into "Primary" (our code decides) vs "Impact" (needs government/users to act too).
- Goal #1 is trust, not detection — it's the thing we actually nailed (0/57 FP).
- "potholes + cracks, expanding to 7" matches reality (2 classes today), no longer contradicts the Detection Classes section.

1b. Where we stand today

Goal	Status	Reality
1. Few false alarms (<5%)	✅ Done	0/57 false positives — best result in the project. ⚠️ only tested on handheld video, never from a moving car.
2. Detect & geo-tag damage	🟡 Half	Works for potholes + cracks (2 of 7). GPS tagging is built (EXIF→GPX) but never tested on a real drive.
3. Merge duplicates	✅ Built	PostGIS clustering works. ⚠️ never tested at real data volume.
4. Upload in <5 sec	❓ Unknown	Built, but never measured.

One-line summary: The *software* for every goal exists, but almost nothing is proven in the real world (moving vehicle, real data volume, a stopwatch). Product built; not yet proven.

1c. Exact tests to close the gaps

Goal	Test to run	What "pass" looks like	Blocker / needs
1. False alarms	Run the model on dashcam footage from a moving vehicle — both roads with no potholes (count false fires) and roads with potholes.	False positives < 5% of frames on the no-pothole footage.	🔴 Need real dashcam footage (this is task A3-5). Current 0/57 is handheld only.
2a. Detection quality	On the same real vehicle footage, measure precision & recall for potholes and cracks.	Agreed bar (see D-4 — the old 95% is falsified; a new honest target is pending).	🔴 Dashcam footage + the D-4 target decision.
2b. Geo-tagging	Do one real recorded drive with a phone/GPX log. Confirm each detection gets the correct lat/long and <code>speed_kmph</code> , and that faked-GPS fallback does not trigger.	Every report's location matches the true track within a few metres; <code>gps_source = exif/gpx</code> , not <code>faked</code> .	🔴 One real drive with a camera + GPS.
3. Duplicate merging	Seed ~ 10k reports (<code>scripts/seed_10k_reports.py</code> exists), or drive the same road twice . Count raw sightings vs final verified issues.	> 80% fewer issues than raw reports for genuinely-duplicate sightings.	🟡 Run the seed script; no external blocker.
4. Upload latency	Time <code>POST /detect-image</code> end-to-end (upload → inference → stored in DB) and record p95 . Repeat over a throttled/4G connection.	p95 < 5 s end-to-end.	🟡 Write a small timing harness; no external blocker.

Note: Goals 3 and 4 have **no external blocker** — they can be tested this week. Goals 1 and 2 are all blocked on the same thing: **real footage from a moving vehicle** (A3-5), which is also the project's single biggest untested risk.

2. Target Users

2a. Agreed rewrite

The PRD's 5 users collapse into **3 real groups** (the first three PRD entries are all the same user):

#	User	Who they are	Role in the product
1	Authorities (municipal corps, public works, highway depts)	Government bodies who fix roads	Use the dashboard to see and repair damage
2	Fleet operators	Companies running many vehicles (bus, taxi, delivery, trucking)	Collect the data — cameras on their vehicles; get bad-road warnings back
3	Admin (internal — <i>missing from the PRD, must be added</i>)	Our own RoadSense team	Create authority/fleet accounts, swap the AI model, monitor system health

Dropped from target users: Citizens → moved to *future idea*, not a launch user (see decision below).

2b. Where we stand today

The app has exactly **3 login roles**, matching the 3 groups above:

User	Built?
Authorities	✓ Full dashboard
Fleet operators	✓ Fleet dashboard
Admin (internal)	✓ Admin dashboard
Citizens	✗ Nothing citizen-facing exists

2c. Decisions locked

1. **Citizens are deprioritised.** Fleets already cover the roads that matter (highways, bus routes, main roads) with cameras already driving all day. Citizens would only add small side-streets, and cost a whole separate phone app + messy/unverified data. **Verdict: fleets + authorities give ~80% of the value for ~20% of the effort. Citizens = "phase 5, maybe."**
2. **Admin is kept, and is internal.** It's the owner's key — without it nobody can even create an authority or fleet account. It is *our team*, not a customer.

2d. Actions (PRD edits, not code)

Action	Why
Rewrite the PRD user list as Authorities / Fleets / Admin (internal)	Collapses the redundant 3 authority entries; reflects what's built.
Add "Admin (internal operator)" to the PRD	It's missing today, but it's a required user.
Move Citizens to a "Future / possible users" note	Honest about scope; leaves the door open without committing.

2e. Open decision

- **Who is the #1 user?** Authority (buys + fixes) vs Fleet (drives + generates data). Not yet decided — it shapes who the dashboard is designed for and who pays. *Leaning: fleets are the data engine, authorities are the buyer — revisit alongside the business model.*

3. System Overview

3a. Verdict on the PRD flow

The PRD's straight-line flow (Vehicle → Camera → AI Detection → GPS+Timestamp → Cloud Upload → Verification → Database → Authority Dashboard → Repair) **mostly matches what's built** — this is one of the more honest sections. Two fixes needed:

1. **It hides the biggest architecture choice** (see decision below): the order AI Detection → Cloud Upload implies detection runs *on the vehicle (edge)*, but the actual build runs detection *in the cloud*.
2. **Small gaps:** the diagram shows only the Authority Dashboard (Fleet + Admin also exist), and omits **image storage (S3/MinIO)** and **notifications** — both built.

3b. Decision locked — Edge vs Cloud detection

Where does the AI run: on the vehicle (edge) or in the cloud?

Lens	Leans
ML engineer — model does ~10 fps vs 30 fps video; sample frames either way	Cloud (improve model without touching cars)
Systems architect — full video over 4G is heavy + drops signal	Edge (upload only the hits)
Cost/business — cheap camera vs cheap running cost	Edge at scale, cloud early
Product/user — fleets want a device that works in tunnels/no signal	Edge
Pragmatic shipper — cloud is already built ; edge = new hardware + model shrinking	Cloud (ships now)

✔ **Decision: Cloud now, edge on the roadmap.**

- **Beta:** detection runs **in the cloud** (already built, flexible, model improvable daily). Upload **sampled frames, not full video**, to keep bandwidth sane.
- **Production/scale:** move to **edge** once fleet size makes cloud bandwidth + dead-zone gaps painful. This is where real dashcam products end up.

One line: start cloud because it's done and flexible; plan for edge because bandwidth will eventually force it.

3c. Actions

Action	Type	Why
Redraw the PRD flow so Cloud Upload comes before AI Detection	PRD edit	Match reality: image uploads first, cloud detects.
Add a note: " <i>Detection is cloud-based for beta; edge deployment is a production/Phase-4 goal.</i> "	PRD edit	Names the future without pretending it's built.
Add Fleet + Admin dashboards, image storage (S3), and notifications to the diagram	PRD edit	All built, currently invisible in the flow.
Implement frame sampling on upload (don't send full video)	Code (Phase 4)	Keeps cloud bandwidth viable; model can't do 30 fps anyway.

4. Hardware

4a. Verdict

The PRD's hardware list (Raspberry Pi 5 / Jetson / etc.) is **edge hardware** — mini-computers to run the AI *inside the car*. But we decided **AI runs in the cloud for beta** (§3b), so **none of that compute gear is needed now**. It belongs in the future (edge) phase.

For the cloud beta, hardware = 3 things, and a smartphone is all three:

Need	Why	A phone provides it
Camera	See the road	✓
GPS	Tag location	✓
Internet upload	Send to cloud	✓ (4G)

So the **beta reference device is a smartphone**. The Pi/Jetson list moves to the edge/production phase.

4b. Notes

- **Accelerometer + gyroscope ("optional")** — genuinely useful *later*: hitting a pothole produces a physical jolt, a strong "this is real damage" signal that could improve severity. **Not built** — severity today is image-only. Keep as a future idea.
- **Too many options**. 3 cameras × 3 computers = 9 combos with no choice made. Pick **one reference device** to build and test against.

4c. Reality check

● **Nothing has ever run on real hardware**. All results are from video *files* on a laptop — no phone, no Pi, no car mount tested. This overlaps with the biggest project risk: **no validation from a moving vehicle** (A3-5).

4d. Actions

Action	Type	Why
State the beta reference device = smartphone (camera + GPS + upload)	PRD edit	Matches the cloud decision; cheapest path.
Move Raspberry Pi / Jetson to a "future edge hardware" note	PRD edit	Not needed for cloud beta.
Keep accelerometer/gyroscope as a future severity-boost idea	PRD edit	Real signal, but unbuilt.
Do one real test on a phone mounted in a car	Test (ties to A3-5)	Nothing has touched real hardware yet.

5. AI Pipeline

5a. Status of each step

PRD flow: Video → Frame Extraction → YOLO Detection → Road Segmentation → Depth Estimation → Severity → GPS Tagging → Cloud Upload

Step	Built?	Note
1. Frame extraction	✓	<code>ai/extract_frames.py</code>
2. YOLO detection	✓	Works — 2 of 7 damage types (pothole + crack)
3. Road segmentation	✗	Not built — and should be removed (see below)
4. Depth estimation	✗	Not built — the one real gap (see below)
5. Severity	✓	Built, but rough — image-only, no true depth
6. GPS tagging	✓	<code>ai/gps.py</code> — EXIF → GPX
7. Cloud upload	✓	Built

5 of 7 done. The two missing steps are very different stories.

5b. Step 3 — Road Segmentation: remove it, don't build it

Its job was to cut false alarms (tree/dashboard mistaken for a pothole). But the team **already tested this**:

- False alarms are already **0/57** — nothing for it to fix.
- It would actively **hurt**: ~38% of real potholes sit in the same zone it would filter out → it would throw away real detections.

✓ **Decision: delete this step from the PRD.** A rare "we proved we don't need it" result. (Reopen only if false positives ever appear on real footage.)

5c. Step 4 — Depth Estimation: the real gap, but risky

Severity today is guessed from the flat 2D image — it **cannot measure how deep** a pothole is, so a shallow dip and a tire-killer can score the same. True depth needs a second model (e.g. MiDaS), which would make

severity genuinely accurate.

The catch: the speed budget is already tight — **93.7 ms vs a 100 ms limit (6% spare)**. A depth model would blow past it. Real upgrade, real cost.

✅ **Decision: worth doing, but NOT a beta item** — it forces a speed/sampling decision first (ties to \$3b frame sampling).

5d. Actions

Action	Type	Why
Remove "Road Segmentation" from the PRD pipeline	PRD edit	Proven unnecessary and harmful (–38% recall).
Note severity is relative (image-based) , not true depth	PRD edit	Honest about what "severity" means today.
Mark Depth Estimation as a post-beta upgrade, gated on the latency budget	PRD edit	Real improvement, but breaks the 100 ms limit.
Update "YOLO Detection" to say 2 of 7 classes today	PRD edit	Matches reality (see Detection Classes section, below).

6. Detection Classes

6a. Reality

The PRD lists **7** classes. Built: **2** (pothole + crack) — and crack barely fires on real footage, so realistically **1.5 of 7**.

6b. The insight — not all 7 are worth building, and two don't belong

- **Speed Breaker** — not damage; it's a *deliberately built* bump. Detecting it is a **navigation** feature, not road-health. Wrong list.
- **Patch Repair** — an *already-fixed* road. The product finds damage to fix; hunting for finished roads is low value.
- **Water-filled Pothole** — really just a pothole with water in it. Better as a **severity flag** than its own class.

6c. Decision — tier the classes

Tier	Classes	Why
✅ Keep (core)	Pothole, Road Crack	Built. Validate crack on real footage.
➡ Build next SOON	Broken Road, Road Edge Damage	Real damage, real value, buildable.
🟡 Downgrade to a flag	Water-filled Pothole	A depth/severity flag on potholes, not a class.
❌ Drop	Speed Breaker, Patch Repair	Not damage / already fixed — wrong for this product.

Target set: 4 real damage classes (pothole, crack, broken road, edge damage) + a water flag.

6d. The hidden cost (why cutting matters)

The model's proven ceiling is **label quality**, not quantity. Every new class = **thousands more hand-labelled images**. So each added class is expensive — the reason to **cut the weak ones** rather than chase all 7.

6e. Actions

Action	Type	Why
Cut the PRD class list from 7 → 4 (+ water flag)	PRD edit	Drop speed breaker + patch repair; downgrade water-filled.
Mark Broken Road, Road Edge Damage as the next classes to train	Roadmap	Highest value-per-effort after the current 2.
Validate the crack class on real footage	Test	It passes on RDD but emits ~zero cracks on real video.
Note: adding classes is gated on better labels , not more data	PRD edit	Ties to the D-4 accuracy reality.

7. Severity

7a. Reality — only 1 of 5 promised factors is used

PRD: 3 levels (Low/Medium/High) based on **diameter, estimated depth, speed, traffic density, repeat detections**.

Factor	Built?
Diameter (size)	● Roughly — guessed from the 2D image, relative not exact
Estimated depth	✗ Not built (needs the depth model — \$5c)
Speed	✗ Captured but not used for severity
Traffic density	✗ Not built at all — no traffic data anywhere
Repeat detections	✗ Not used for severity

Promise = rich 5-factor score. Build = **one rough size guess**.

7b. The insight — the 5 factors measure 3 different things

They're conflated. Really they answer:

1. **How bad is the hole?** → size + depth → *this is true severity*
2. **Are we sure it's real?** → repeat detections → *confidence/verification*
3. **How urgent to fix?** → traffic density + repeats → *priority*

A small pothole on a busy highway isn't *more severe* — it's *more urgent*. And **speed doesn't belong at all**: how fast you drove past doesn't change the hole; it only affects photo quality (fast = blurry).

7c. Decision

- **Severity = how bad the hole is** → keep it **size now, depth later**. Nothing else.
- **Priority = how urgent to fix** (busy road + many sightings) → make it a **separate score**. This is what authorities actually act on.
- **Drop speed** as a severity factor.

7d. Actions

Action	Type	Why
Redefine severity in the PRD as size (+ depth later) only	PRD edit	Honest; matches what severity can actually measure.
Add a separate "Priority" score (traffic + repeat sightings)	PRD edit + future code	It's a different question, and the one authorities care about.
Remove speed from severity factors	PRD edit	Irrelevant to how bad the hole is.
Note severity is currently relative, not metric (needs per-camera calibration)	PRD edit	Already flagged in the model registry weaknesses.

8. Verification

8a. Reality

Built, and one of the strongest areas. Reports within **20 m** of each other are clustered into a single issue (PostGIS `ST_DWithin`). This is the moat — it kills duplicate reports.

8b. The problem — "verified" is overclaiming

Today **one single report creates an "issue."** Clustering removes *duplicates*, but nothing *proves* the pothole is real — one car glancing at a shadow still becomes an "issue." "Verified" currently means "seen once," which isn't verification.

8c. Decision — add a sightings threshold

Sightings	State	Meaning
1	Unverified / pending	Maybe real, maybe a fluke
2+ (different vehicles)	Verified	Multiple cars saw it — trust it

This makes "verified" mean something, and plays to the fleet strength (many cars = many confirmations).

8d. Notes

- **20 m radius untested at scale** — two *different* potholes ~15 m apart on the same road could wrongly merge into one. Check during the performance test (B3-7).
- **Never tested at real data volume.**

8e. Actions

Action	Type	Why
Add a "verified" threshold (≥ 2 sightings from different vehicles)	Code + PRD edit	Makes "verified" real; leverages fleet coverage.
Show unverified vs verified distinctly on the map/dashboard	Code (frontend)	Authorities should know what's confirmed vs pending.
Test the 20 m radius at ~10k reports (does it over-merge?)	Test (B3-7)	Nearby-but-distinct potholes must not collapse into one.

9. Data Captured

9a. Reality — 9 of 10 fields captured

Solid section. PRD list: Report ID · Vehicle ID · Lat/Long · Timestamp · Speed · Road Name · Image/Video · Confidence · Severity · Weather — almost all genuinely stored and working.

9b. Three fixes

1. **Weather — captured but used nowhere.** Stored in the DB, but no dashboard/popup/filter reads it. Dead weight today. **Decide: use it or label it "future."**
 - Keep only if it feeds something real (rain → potholes worsen; future degradation prediction).
 - **Lean: keep collecting, but mark "for future use"** — don't call it a feature until something reads it.
2. **"Image / Video" → "Image."** Only still frames are stored; video is never kept.
3. **Add GPS source to the list** — already captured (real/faked provenance, shown as amber warning pins). A genuine trust feature that's missing from the PRD's field list.

9c. To add later

If the verification threshold (§8c) is adopted, also store **sightings count** and **verified status** per issue.

9d. Actions

Action	Type	Why
Decide weather's fate — surface it or label "collected for future use"	PRD edit (+ maybe frontend)	Stop it being an invisible dead field.
Change "Image / Video" → "Image" in the PRD	PRD edit	Video isn't stored.
Add GPS source to the captured-fields list	PRD edit	Already built; it's a trust win.
Plan sightings count + verified status fields	Future code	Needed once verification threshold lands.

10. Backend Stack

10a. Reality — basically correct

Everything listed (Next.js + Tailwind · FastAPI · Python + PyTorch + YOLO · PostgreSQL + PostGIS · S3 · Redis) is genuinely what's running. Rare and good. Three small fixes only.

10b. Fixes

1. **Wrong name.** It's titled "Backend Stack" but line 1 is the *frontend*. It's the whole **Tech Stack** — rename.
2. **The map is missing.** The product is built around **Leaflet + OpenStreetMap** — core, and not listed.
3. **Invisible-but-important tools not mentioned: Docker** (everything runs in it), **Alembic** (DB migrations); minor: **S3 = MinIO locally**, real S3 in production.

10c. Actions

Action	Type	Why
Rename section "Backend Stack" → "Tech Stack"	PRD edit	It covers frontend too.
Add Leaflet / OpenStreetMap	PRD edit	The map is central to the product.
Add Docker and Alembic ; note S3 = MinIO in dev	PRD edit	Real parts of the running system.

11. Dashboards

11a. Reality — all 11 items built (strongest section)

Every listed feature exists and works: Authority (live map · severity · pending reports · repair tracking · analytics), Fleet (vehicle status · detection history · camera health), Admin (user mgmt · AI model mgmt · system monitoring). Cleanly matches the 3-user model (§2) — and correctly **no citizen dashboard**.

11b. Three gaps

1. **Notification bell missing.** Backend notifications are fully built, but **no bell in the UI** — so high-severity alerts are computed and never seen. (Consistent with §B3-2 correction.)
2. **Weather not shown** — same dead field as §9.
3. **Never tested at real data volume** — analytics + road-health screens could be slow at 10k reports (B3-7).

11c. Deeper point — sort by priority, not just severity

Dashboards show **severity** ("how bad"), but authorities act on **priority** ("how urgent" = bad + busy road + repeat sightings — see §7). "Pending reports" isn't ranked by urgency. Once the priority score exists (§7c), the Authority dashboard should **sort by it** — that's a decision tool vs a list.

11d. Actions

Action	Type	Why
Build the notification bell in the UI	Code (frontend)	Backend is done; alerts are currently invisible.
Decide weather display (with §9)	PRD/frontend	Same dead field.
Rank pending repairs by priority once the priority score lands	Code (frontend)	Turns a list into a decision tool.
Load-test analytics + road-health at ~10k reports	Test (B3-7)	Confirm the screens stay fast.

12. Repair Workflow

12a. Reality — code matches the PRD exactly

All 7 states are built (`VALID_TRANSITIONS` in `main.py:986`): `detected` → `verified` → `assigned` → `inspection` → `repair` → `completed` → `closed` . It's a *proper* state machine with real-life detours: false reports close early, failed repairs bounce back, returned potholes reopen a closed issue. Honest and done.

12b. Important flag — "Verified" means two different things

Word collision with §8:

- **Auto-verified** (§8) = system clustered 2+ sightings → confidence it's real.
- **Workflow "Verified"** (here) = a human approved it for repair.

Same word, two meanings. **Rename the workflow step** (e.g. **"Approved"** or **"Triaged"**) to avoid the clash.

12c. Small question — Completed vs Closed

What's the difference? Likely "Completed = road fixed" vs "Closed = admin/paperwork done." Defensible, but confirm a real authority needs both — otherwise merge them.

12d. The real gap

A beautiful state machine **no real authority has ever driven**. Every transition passes in tests, but nobody has run a genuine repair through it — so it's unproven that these 7 steps match how a real city works.

12e. Actions

Action	Type	Why
Rename the workflow "Verified" step (→ Approved/Triaged)	Code + PRD edit	Removes the clash with auto-verification (§8).
Confirm or merge Completed vs Closed	PRD edit	Avoid a redundant step.
Walk a real authority through the workflow	Validation	Confirm the 7 states match a real municipal process.

13. APIs

13a. Reality — all 7 exist , but the list is stale

The 7 listed endpoints (`/detect` , `/upload` , `/reports` , `/map` , `/analytics` , `/verify` , `/repair`) all work. But the backend actually has **~25+ endpoints** — the list reflects the early phases and **omits whole Phase-3 features**:

- **Login / auth** — the entire security model
- **Fleet** endpoints — a whole dashboard
- **Notifications** — a whole feature
- **Admin / model registry** — a whole feature
- **/health** — monitoring

13b. Decision

A PRD doesn't need every route (that's API docs). **Group by feature** instead of listing 25: Auth · Detection/Upload · Reports/Map/Analytics · Verify · Repair · **Fleet** · **Notifications** · **Admin/Models** · **Health**.

13c. Actions

Action	Type	Why
Rewrite the API list grouped by feature (add Auth, Fleet, Notifications, Admin, Health)	PRD edit	Reflects the real product, not just early phases.

14. Security

14a. Status — 3 solid, 2 shaky

Item	Status
JWT authentication	 Done
Audit logs	 Done
Role-based access	 Done
Encrypted uploads	 Prod-only — dev uploads unencrypted (<code>s3_storage.py:39</code>)
GPS validation	 Built but was silently broken for months (UTC bug, G-14); never run on real data

14b. Insight 1 — a broken-but-present check is worse than a missing one

The teleportation guard (blocks fake GPS jumps) was **inert on every non-UTC server** until recently — you trusted a check that wasn't running. Fixed now, but **never exercised on real data**. Don't assume it works; test it for real.

14c. Insight 2 — the big missing item: privacy

The product photographs **public roads**, capturing **faces and licence plates** = personal data, and a legal issue under privacy law in real deployment. The PRD says nothing about it. Need to **blur faces and plates**

before storing images. Genuine gap.

14d. Also missing — login protection

No **rate-limiting** on login → password-guessing is unthrottled.

14e. Actions

Action	Type	Why
Test the GPS teleportation guard on real data	Test	It was silently off for months; unproven.
Enable/verify upload encryption end-to-end (not just prod flag)	Code	Currently dev-unencrypted.
Add privacy to the PRD — blur faces + licence plates before storage	PRD edit + code	Legal requirement for street imagery.
Add login rate-limiting	Code	Stop brute-force password guessing.
Finish SECRET_KEY rotation for production	Ops	Already hard-fails without env var; document the rotation.

15. Functional Requirements

15a. Status — mostly met

Requirement	Status
Real-time detection	🟡 Stretch — ~10 fps vs 30 fps video; <i>near</i> -real-time, not real-time
GPS tagging	✅ (unproven on a real drive)
Cloud upload	✅
Duplicate merging	✅ Strong
Notifications	🟡 Backend done, no bell in the UI
Dashboard	✅
Repair workflow	✅

15b. Actions

Action	Type	Why
Stop calling it " real-time " — say "near-real-time (~10 fps)"	PRD edit	Honest about the frame rate.
Build the notification bell (dup of §11d)	Code	Backend done, UI missing.

16. Non-functional Requirements — the honesty crunch

16a. Status — 1 won, 1 false, 3 untested

Requirement	Status
False positive rate <5%	✅ Smashed — 0% on real footage. Best result.
Detection latency <100 ms	🟡 Barely — 93.7 ms, 6% spare, measured on a laptop not a real device
Detection precision >95%	🔴 Falsified — proven <i>unreachable</i> by scaling (D-4)
API uptime 99.9%	? Never measured — no real deployment, no monitoring
Dashboard load <2 s	? Never load-tested at real data volume

16b. The big recommendation — re-baseline on measured numbers (this is D-4)

- **Keep** FP rate <5% — earned.
- **Replace** "precision >95%" with an honest, measured target (e.g. "detects X% of potholes on real dashcam footage").
- **Prove or drop** uptime, dashboard load, and real-device latency — an unmeasured number is a hope, not a requirement.

16c. Actions

Action	Type	Why
Renegotiate precision >95% → an honest measured target	PRD decision (D-4)	Current target is provably false.
Measure dashboard load at ~10k reports	Test (B3-7)	Confirm <2 s or restate it.
Measure latency on the real device (phone), not a laptop	Test	Laptop 93.7 ms ≠ phone latency.
Define how uptime is measured (needs a real deployment + monitoring)	Ops/Phase 4	Can't claim 99.9% with nothing deployed.
Keep FP rate <5% as the headline win	PRD edit	The one target genuinely beaten.

17. Future Features

17a. Two problems

Road Health Score · Flood Detection · Bridge Crack Detection · Traffic Sign Detection · Accident Detection · Smart City Analytics

1. **Road Health Score is misfiled — already built** (backend + choropleth, §B3-3). Move it out of "future."
2. **Some aren't this product.** Traffic Sign Detection + Accident Detection are *different products* (navigation / real-time safety), not road damage. Keep, but mark clearly "someday / maybe."

17b. Actions

Action	Type	Why
Remove Road Health Score from Future (it's done)	PRD edit	It's a shipped feature.
Mark traffic-sign / accident detection as out-of-scope "maybe"	PRD edit	Different products; avoid scope creep.

18. Development Roadmap

18a. Reality — 4 empty labels

Phase 1 PoC · Phase 2 MVP · Phase 3 Beta · Phase 4 Production — no "done means what," no dates. No way to tell when a phase is finished. Meanwhile **CLAUDE.md is extremely detailed** — the PRD roadmap is a hollow version of it.

18b. Actions

Action	Type	Why
Give each phase a one-line "done when..."	PRD edit	Makes phases checkable.
Pull real status from CLAUDE.md (currently mid-Phase-3, mostly done)	PRD edit	Reflect actual progress.

19. Success Metrics — a duplicate section

19a. Reality — repeats Non-functional Requirements

Precision ≥95% · Recall ≥90% · Upload latency <5s · Duplicate reduction >80%

Precision ≥95% appears **twice** in the PRD (here + §16), falsified both times.

Metric	Status
Precision ≥95%	● Falsified
Recall ≥90%	● Unproven, likely unreachable
Upload latency <5 s	? Never measured
Duplicate reduction >80%	? Built, never measured

19b. Actions

Action	Type	Why
Merge Success Metrics into Non-functional Requirements	PRD edit	One metrics section, not two duplicates.
Re-baseline with the D-4 honest numbers	PRD decision	Same falsified targets as §16.
Measure upload latency + duplicate reduction	Test	Both unmeasured (ties to Goals §1c).

20. Long-Term Vision

20a. Reality — good, one tweak

A continuously updated *digital twin* of road infrastructure from *crowdsourced* vehicle data → predictive maintenance.

Solid, motivating vision. One tension: it says "**crowdsourced**," but the data model is **fleets, not crowds** (§2 — citizens dropped).

20b. Action

Action	Type	Why
Change " crowdsourced " → " fleet vehicle data "	PRD edit	Match the actual data model.

✅ Walkthrough complete — all 20 sections reviewed.

Master Action List

Everything from the walkthrough, gathered and grouped. Four buckets: **Decisions**, **PRD edits**, **Code**, **Tests/Validation**.

● Big decisions (blockers — need a human call)

#	Decision	From
D1	Re-baseline accuracy targets — precision $\geq 95\%$ / recall $\geq 90\%$ are falsified. Set an honest, measured target. (<i>This is D-4.</i>)	§16, §19
D2	#1 user: Authority or Fleet? Shapes dashboard design + who pays.	§2
D3	Cloud now, edge later — confirmed direction; write it into the PRD.	§3
D4	Class scope: 7 → 4 (pothole, crack, broken road, edge damage) + water flag; drop speed breaker + patch repair.	§6

PRD edits (documentation — cheap, do in a batch)

- Goals: make measurable, split "primary (we control)" vs "impact"; lead with trust/FP-rate. **(§1)**
- Target Users: collapse to Authorities / Fleets / Admin; add Admin (internal); move Citizens to "future". **(§2)**
- System Overview: put **Cloud Upload** before **AI Detection**; add Fleet+Admin dashboards, S3, notifications; note cloud-for-beta/edge-later. **(§3)**
- Hardware: beta device = **smartphone**; move Pi/Jetson to "future edge"; keep accel/gyro as future idea. **(§4)**
- AI Pipeline: **remove Road Segmentation**; mark depth as post-beta; say severity is relative; YOLO = 2 of 7 classes. **(§5)**
- Detection Classes: cut 7 → 4 (+water flag); note classes gated on better labels. **(§6)**
- Severity: redefine as **size (+depth later)**; add separate **Priority** score; drop speed. **(§7)**
- Data Captured: decide weather (use or "future"); "Image/Video" → "Image"; add **GPS source**. **(§9)**
- Backend Stack: rename → **Tech Stack**; add Leaflet/OSM, Docker, Alembic. **(§10)**
- APIs: regroup **by feature** (add Auth, Fleet, Notifications, Admin, Health). **(§13)**
- Security: add **privacy** (blur faces/plates). **(§14)**
- Requirements: stop saying "real-time" → "near-real-time (~10 fps)". **(§15)**
- Future Features: remove Road Health Score (done); mark traffic-sign/accident as out-of-scope. **(§17)**
- Roadmap: add a "**done when...**" line per phase; sync status from CLAUDE.md. **(§18)**
- Success Metrics: **merge into Non-functional** (it's a duplicate). **(§19)**
- Vision: "crowdsourced" → "fleet vehicle data". **(§20)**

Code (build work)

#	Item	From	Notes
C1	Notification bell UI	§11, §15	Backend done; alerts currently invisible.
C2	Verification threshold (≥2 sightings = "verified") + show verified vs pending	§8	Makes "verified" real.
C3	Rename workflow "Verified" → Approved/Triaged	§12	Clashes with auto-verification.
C4	Priority score (bad + busy road + repeats) + sort dashboard by it	§7, §11	The number authorities act on.
C5	Frame sampling on upload (not full video)	§3	Keeps cloud bandwidth viable.
C6	Privacy: blur faces + licence plates before storage	§14	Legal requirement.
C7	Login rate-limiting	§14	Stop brute-force.
C8	Decide + wire weather display (or drop the column)	§9, §11	Currently a dead field.
C9	Depth model for real severity (post-beta)	§5, §7	Blows the 100 ms budget — gate on sampling.

Tests / Validation (prove what's built)

#	Test	From	Blocker
T1	Real dashcam footage from a moving vehicle — FP rate, precision, recall	\$1, \$4	● Need footage (A3-5) — biggest risk.
T2	One real recorded drive — GPS tagging correct, not faked	\$1	● Need a drive.
T3	Duplicate reduction — seed ~10k or drive same road twice; measure >80%	\$1, \$8	● Seed script exists.
T4	Upload latency p95 <5 s (incl. 4G)	\$1, \$16	● Write timing harness.
T5	Dashboard + road-health load test at ~10k reports (<2 s)	\$11, \$16	● No blocker (B3-7).
T6	20 m clustering radius — doesn't over-merge distinct potholes	\$8	● Part of T5.
T7	GPS teleportation guard on real data — was silently broken	\$14	● No blocker.
T8	Latency on the real device (phone), not a laptop	\$16	Ties to T1.
T9	Walk a real authority through the repair workflow	\$12	Needs a real user.

The theme across the whole audit: the *software is built*, but almost nothing is *proven in the real world*. The single highest-value unlock is **T1/T2 — real footage from a moving vehicle** — it unblocks the accuracy re-baseline (D1) and the biggest untested risk at once.